



University of
Nottingham
UK | CHINA | MALAYSIA

SCHOOL OF COMPUTER SCIENCE

UNIVERSITY OF NOTTINGHAM

Automatic Music Transcription for Guitar: A Machine Learning Approach to Creating a Guitar Practice Tool

Interim Report

Author:

Aryan Sunil

Student ID:

20547604

Course:

COMP3003

Program:

BSc Hons Computer Science
with Artificial Intelligence

December 8, 2025

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Aims and Objectives	3
2	Related Work	4
2.1	Pitch Estimation Methods	5
2.2	Fretboard Mapping Approaches	5
2.3	End-to-End Vs Sequential Pipelines	5
2.4	Technique Classification	6
2.5	Datasets	7
2.6	Music Segmentation	7
3	Implementation	8
3.1	Guitar Audio Isolation	8
3.2	Pre-Processing Pipeline	8
3.3	Post-Processing Pipeline	9
3.4	Fretboard Mapping	9
3.5	Fretboard CNN	10
3.6	Tablature Generation	11
4	Progress and Evaluation	12
4.1	Pitch Estimation Evaluation	12
4.2	Fretboard Evaluation	12
5	Project Management and Reflection	13

5.1	Project Plan	13
5.2	Meetings Records	14
5.3	Contributions and Reflections	15
5.4	Legal, Social, Ethical and Professional Issues	16
5.5	Acknowledgements	17
	Bibliography	18

1 Introduction

1.1 Background and Motivation

The availability of guitar tablature (musical notations for stringed instruments such as a guitar) for specific composition highly depends on the song's popularity. If there is no tab available for the song, the only alternatives are to learn the song by ear, which is challenging and requires a high level of expertise or to pay a professional transcriber. Manual tabs created also come under a substantial scrutiny with the accuracy, as users may publish unverified or inaccurate tabs [1]. The inaccuracy can be a significant barrier for entry-level self-taught learners, leading to frustration and hindering their progress [2]. This research proposes the development of an intelligent guitar practice tool using automatic music transcription to solve the common struggles users face.

An Automatic Music Transcription (AMT) is a tool used to convert a music signal into musical notation that can be read by musicians to play their instrument [3]. In the context of this study, the goal is to convert guitar play into guitar tablature, i.e. music notation for documenting frets and string instruments. Building an AMT is a challenging task that includes multiple sub-tasks that need to be handled, including: Onset Detection, Pitch Detection, Note Tracking, Handling polyphonic audio [4], Effects and various guitar techniques [5]. Leading transcription systems are still significantly below the quality of a human expert [3].

While AMT may not perform better than a music expert, the progress it has made suggests it can be a substitute solution to the problem and can still be used by users to correct any inconsistencies in the tab generated or be used as an assistive educational tool to allow users to confirm if the tab that they created is correct. As an intelligent guitar practice tool, it will include segmentation of tab generation to practice sections of the song, improving users' development in learning a tab. The option of exporting to industry-standard format gives users the option to use their preferred tool [6].

Index Terms: Onset Detection, Note Tracking, Polyphonic Audio

1.2 Aims and Objectives

The primary aim of this research is to be able to design and evaluate an assistive practice system for guitarists, by integrating AI and pitch detection with features including segmentation and fretboard mapping to create a functional aid. To achieve this, an End-to-End system and a Sequential Transcription Pipeline will be tested to compare the frameworks to determine which methodology would best balance pitch accuracy with the practical educational utility required for practice, specifically optimising ergonomic playability of the generated tabs.

In order for this project to be deemed successful, the main objectives have been established as follows:

1. Research previous AMT approaches (Fretting Transformer [7], TabCNN [8]) and benchmark the models used to compare effectiveness of pitch detection accuracy, fretboard mapping performing using a dataset.
2. Develop a sequential process pipeline: 1. pitch detection model using Basic Pitch [4] to convert audio to midi, 2. Various fretboard mapping models namely simple heuristic, CNN context based solution.
3. Develop a software-based user interface, implementing options to export, slicing sections of tabs to segments and converting tab generation to be exported to industry-standard format (MusicXML [9], .g5 Guitar Pro, PDF) to enable compatibility with existing guitar tab software (e.g. MuseScore, Guitar Pro)
4. Generate guitar technique classification (bends, pull-offs, etc) from audio feature. Training the model using SynthTab's [10] dataset and GOAT dataset [11] evaluating novel features of the tool.
5. Evaluating the system's performance (monophonic audio - single guitar note, polyphonic audio - multiple guitar notes, multi-instrument polyphony audio - mix band audio), evaluating with various guitar styles (finger style, pick, tapping, legato). Measuring model evaluation metrics through F1-score, Recall, Precision, string/fret accuracy, and technique detection accuracy. Comparing findings with other models (TabCNN [8] and heuristic baseline). Targeting to achieve a F1 score over 0.8.
6. Conduct user testing with 5-10 guitarists from different skill level (beginner to expert) to find real usage performance, playability of tabs, ability to learn efficiently, users ratings and preferences [2].

2 Related Work

An Automatic Music Transcription (AMT) tool has been a concept that has remained a subject of significant interest for researchers and musicians for a long time. There exist complete solutions similar to the proposed system, such as Songsterr AI tab generation. These show the advanced leaps in progress that have been made in this field. Due to it not being feasible to compare the proprietary system, the model will be benchmarked against open-source AMT research solutions. The development in research of AMT has shifted from building heuristics (e.g A* search) to implementing deep learning to create sophisticated models to handle the nuances and challenges of music processing. Due to the nature of polyphonic

audio and challenges such as pitch redundancy, multiple approaches to this problem have been made. One of the methods that has been explored includes using hexaphonic pickups to generate transcriptions by analysing signals in each individual string.

2.1 Pitch Estimation Methods

A popular option is to use a probabilistic approach like the pYin algorithm [12], which is an extension of Yin algorithm [13] to solve the limitation during post-processing (cleaning raw audio) where it cannot fall back on alternative interpretations of the signals. pYin overcomes the problem by outputting multiple pitch candidates with associated probabilities, reducing the loss of useful information. The core reason why pYin would not be an ideal option is due to being a monophonic transcription tool, while an AMT would require a multi-instrument polyphony transcription rather. A deep learning solution includes CREPE [14] designed for single-f0 estimation and effectively fails to transcribe polyphonic audio.

Basic Pitch being lightweight, it can operate in low-resource settings, making the tool accessible to a much wider range of users. While being lightweight, it still outperforms other strong instrument-agnostic baseline models across nearly all datasets. Basic Pitch performance can be impacted when tested with a full band mix (multiple instruments playing, i.e. songs), as it was designed for polyphonic recordings of a single instrument class. Basic Pitch begins by converting the audio input to a Constant-Q Transform (CQT). Using a technique called harmonic stacking, it is able to capture harmonic-related information. This works by copying and vertically shifting the CQT representation by the number of frequency bins corresponding to each harmonic. The model is trained using binary cross-entropy of the three outputs (pitch, noise, onset). Therefore, improving the note accuracy.

2.2 Fretboard Mapping Approaches

Fretting transformer [7] is a pitch-to-tab process to convert MIDI to guitar tablature, tackling complex issues such as pitch redundancy. It uses an encoder-decoder model utilising the T5 transformer architecture (Text-to-Text Transfer Transformer) [15] to perform tasks including sequence-to-sequence mapping (midi note to guitar tablature), fingering prediction, while ensuring context is taken into account.

2.3 End-to-End Vs Sequential Pipelines

There are two strategies for creating an AMT. This includes an End-to-End system or the Sequential pipelines.

An end-to-end system such as TabCNN [8]. Rather than the sequential options explored previously, an End-to-End approach attempts to solve both the steps, including audio to midi and midi to tablature, by transcribing directly into tablature without an intermediate step like midi conversion. From processing the audio spectrogram, they are able to access more music context, theoretically allowing them to distinguish between the same pitch played on different strings based on timbre differences, solving the problem of pitch redundancy. TabCNN is a convolutional neural network (CNN) designed to estimate individual strings/frets from the audio. TabCNN performed well on the GuitarSet dataset, achieving a Tablature Disambiguation Rate over 89%, identifying pitches that were assigned to the correct fingering. Since it was trained on the GuitarSet, TabCNN does not take into account various guitar techniques, which would be a critical flaw when providing it as a tool for users. It also mentioned adding temporal smoothing to improve performance as future work. The recent architecture, like MT3 [16] that has been proposed, includes using transformers for its multi-transcription functionality, advancing End-to-End systems capabilities.

Technique Aware Audio to Tab Representation Tool (TART) [17] is a hybrid attempt to bridge the gap by including four stages to transcribe audio to tab. The model’s goal is to be able to classify the range of guitar techniques and handle the case of pitch redundancy. The stages include: 1. fine-tuning a piano transcription model which uses a Convolutional Recurrent Neural Network (CRNN) to train on a guitar dataset, 2. Use an MLP-based classifier to extract feature vectors from the audio and update the note events with a technique label. 3. Using a transformer architecture inspired by the fretting-transformer approach to deal with the pitch redundancy confusion to predict the string and fret combination for each note. 4. Merge the data information of the 3 processes to create an ASCII guitar tablature. One of the limitations encountered by TART was the issue of overfitting to the training model, and this was seen when training the model with the Magcii guitar dataset and testing on another (IDMT) dataset. One approach to improve the model would be to train the model on a significantly larger dataset and to diversify the data.

2.4 Technique Classification

Guitar techniques play a major role in achieving realistic tablature. There exist expression styles executed with the fretting hand, such as bends, slides and vibrato, and there are legato techniques like hammer-ons and pull-offs contributing to the continuity of the musical phrase. These multiple factors make guitar technique classification a continuous obstacle. These are: complexities inherent in the guitar audio signal, the difficulties in annotations, and the limited availability of diverse training data. Requiring knowledge in music theory to be able to analyse the acoustic and timbral features of the audio. For example, to figure out expression styles, it requires analysing the inharmonicity (frequency shifts of partial vibration) to differentiate between which string a pitch was played on.

In the early stages, a heuristic approach was carried out, and later, using feature extraction paired with a classifier like Support Vector Machine (SVM) [5]. The modern approach to the problem is to use deep learning techniques such as Technique-Aware Audio-to-Tab (TART) [17], which integrates a Multi-Layer Perceptron (MLP) classifier into its pipeline to amend note events with labels such as "bends" or "vibrato". A model that joins pitch detection and guitar technique classification is TENT [18] using a CNN architecture to perform playing technique detection. Although these models currently struggle to pick up legato techniques, it will be a challenge that needs to be tackled.

The bottleneck to the performance of these classification models is the diversity of the training data. Datasets like GuitarSet provide annotations to fret and string, but they do not label the guitar techniques. Newer datasets, including GOAT, aim to solve this problem by providing a larger dataset of annotated frets and strings, adding guitar techniques, and offering a more robust solution to train models.

2.5 Datasets

Developing a robust AMT tool is fundamentally dependent on the dataset used to train the model. One of the commonly used options includes GuitarSet [19], which contains annotated, 360 acoustic guitar recordings utilising the hexaphonic pickup setup that was mentioned. Since it uses an acoustic guitar, it would not perform as well for electric guitar transcription, and the audio, being a total size of 180 minutes, can lead to issues with overfitting. SynthTab was developed to address the scarcity of annotated guitar data by using synthesised audio, but this leads to it failing to capture the subtle nuances that occur when a user plays the guitar. The large scale of the dataset remains crucial for training the models and mitigating the overfitting issue. Finally, another dataset that is worth looking at is GOAT (Guitar On Audio and Tablature), which focuses on recording electric guitar audio and comprises 5.9 hours of direct input recordings from a variety of guitars and players. Since it contains covers of popular songs, it is not publicly accessible and requires requesting access for research purposes only.

2.6 Music Segmentation

Music segmentation aims to improve practice for users by separating the tabs into its musical sections which are intro, verse, chorus and bridge. This is known as Audio-Based Music Structure Analysis (MSA) [20]. MSA methods assume musical homogeneity, and conclude boundaries as sudden changes in musical attributes and to take repetitions in musical structure into consideration. MSA can be carried out using probabilistic models such as Hidden Markov Models [21] or Matrix Factorisation techniques to simultaneously

determine the precise boundaries and assign labels based on musical similarity. A modern approach uses deep learning models such as CNNs by training as binary classifier directly to the spectral representation to automatically learn and detect musical boundary locations with high temporal accuracy by leveraging patterns found in audio features [22]. These models achieve improved performance on benchmarks like SALAMI [23].

3 Implementation

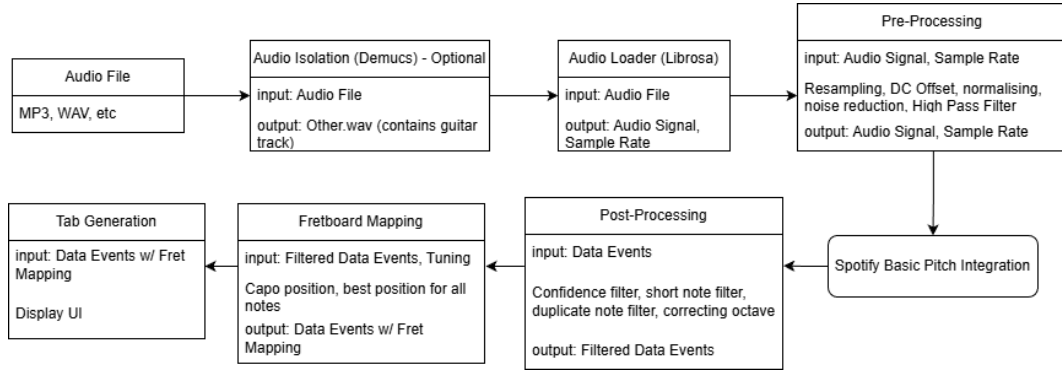


Figure 1: Process Pipeline

3.1 Guitar Audio Isolation

The optional process of Guitar Audio Isolation is for polyphonic audio that includes music containing more than just guitar playing, such as vocals, drums, etc, to be separated into an isolated instrument track. This process utilises Demucs, an open source audio isolation tool from the Facebook research team. The MDX Extra and MDX Extra Q model were evaluated for this purpose. Both models create isolated tracks in the form of vocals, drums, bass and others. The guitar track is stored in other.wav, but it also leads to the issue of other instruments being included as well, so the performance can vary based on the audio. The difference between the two is that the MDX extra Q is a faster model but leads to a cost in quality, while the MDX extra takes a longer time to process.

3.2 Pre-Processing Pipeline

The purpose of pre-processing the audio signal is to filter and clean the signal for accuracy when put into the pitch detection. In this case, it is through Spotify’s Basic Pitch. The first line of processing is to resample the audio to set the sample rate at 22050Hz, since basic pitch internally resamples it before processing as it might affect the accuracy of the Basic Pitch Model. DC offset moves the audio wave to zero amplitude to prevent any premature

clipping of peaks. The audio is then normalised, which is where the audio signal amplitude is scaled so that it meets a target level. The goal of normalising the audio here is to set the audio signal to be consistent.

Basic Pitch is a lightweight, instrument-agnostic model for AMT responsible for generating midi of the frequency audio through a lightweight convolutional neural network with a shallow architecture to gain data, including notes (pitch, onset, offset) and multi-pitch to be then used to find musical information. To circumvent the issue, the audio is isolated through Demucs. However, this introduces a significant computational cost to generate the isolated guitar track, and this will need to be taken into account when building a tool for users. Basic Pitch was chosen over the other possible option due to its ability to transcribe polyphonic signals and based on the Basic Pitch research paper [4] it outperforms competitors including Technique-Embedded Note Tracking (TENT) [18] which is a guitar specific transcription tool where TENT achieved a F-measure (Fno) of 0.763, compared to Basic Pitch's baseline 0.840.

3.3 Post-Processing Pipeline

Once the Basic Pitch returns the data events, it goes through the post process to ensure that accurate tablature can be created. First, this is done by filtering based on a confidence threshold, this ensures that only notes that the basic pitch has a high confidence to be kept. The next step is the short note filter that takes the duration of the note (offset-onset) and checks to see if it crosses the required duration threshold. The notes are then sorted in ascending order based on the onset time so that the correct order of notes is on the tabs. Duplicate notes are filtered based on whether the notes duplicate before the duration threshold. Finally, it goes through octave error correction, which is to match the octave to the constraints of the guitar range.

3.4 Fretboard Mapping

One of the main challenges that differentiates piano transcription from guitar is the ability to play the same note in various positions of the fretboard. Fretboard mapping gives the user options with tunings, allowing users to select the type of tuning from a list, so the tab provided will be tailored. The MIDI number is predefined for each tuning. Another feature implemented is to take into account for capo position. As the capo number rises, the pitch goes down by one semitone.

Let $P = \{p_1, \dots, p_n\}$ be a sequence of detected MIDI pitches and $T = \{(s, m_s)\}$ be the guitar tuning, where $s \in \{1, \dots, 6\}$ is the string number and m_s is the open string MIDI pitch.

For each pitch p_i , we generate candidate positions:

$$C_i = \{(s, f) \mid f = p_i - m_s, 0 \leq f \leq 24\}$$

We select the best position using a greedy strategy:

$$\phi_i = \begin{cases} \arg \min_{(s, f) \in C_i} |f - 5| & i = 1 \\ \arg \min_{(s, f) \in C_i} d(\phi_{i-1}, (s, f)) & i > 1 \end{cases}$$

where $d((s_1, f_1), (s_2, f_2)) = |s_1 - s_2| + |f_1 - f_2|$ is the Manhattan distance.

The fretboard mapping uses a greedy algorithm to choose the local optimal choice as the next step, where it uses the nearest neighbour heuristic to find the next closest to the previous string and fret. The model achieves this by finding all possible positions using the function `map_note_to_fret` to calculate fret for each string, keeping only valid fret positions and listing the candidates. `best_position` aims to select a position close to the middle of the fretboard (5th fret), as we assume that most users play in the middle of the fretboard. If there is a previous note, we take this into consideration and calculate the minimum distance to the next note. Using pathfinding, where it treats each note as a node and transitions as edges. This results in the issue of the fretboard mapping not containing any musical context, not looking ahead, and not being aware of chord shapes. This approach does not guarantee it can simulate how a human would play. The heuristic was chosen as a baseline so to strictly evaluate the importance of context in comparison to see how it improves accuracy. It was designed to be a Minimum Viable Product (MVP) model with the simple structure to simulate how a beginner might attempt to play the guitar by picking the closest fret.

Furthermore, both the approaches fretboard mapping heuristic and fretboard CNN both are within a sequential pipeline mitigating the risk associated with end-to-end systems (like TabCNN). Where, unlike an end-to-end system where a failure in audio processing can result to a total failure in the transcription. The current approaches decouples the two process, to ensure even if there are errors during the pitch detection, it will still be capable of generating tablature.

3.5 Fretboard CNN

An alternative approach to mapping MIDI to the fretboard was developed using a temporal 1-D Convolutional Neural Network. A 1-D CNN was specifically chosen because music is a time-series data (temporal), which means it is able to understand neighbour relationships. Compared to the simple heuristic, the CNN here gains context on the melodic patterns and position on the fretboard from training the model with the SynthTab dataset. It does this by having a context window of 5 (5 before, current, 5 after), giving an input sequence length

of 11, extracting data from the jam files and individual strings to train the patterns in the fret and string played. The MIDI number is embedded in the vector so the model can learn the semantic relationship between notes. The current model uses 3 blocks generating a joint classification model with 150 classes to predict the string and fret from the extracted features. It utilises a channel depth of 32, 64, 128, to 256 to use channel expansion as the channel size increases to capture complex music features of the data. The rectified linear unit (ReLU) activation function applies nonlinearity to the model and dropout is set to 0.25 to prevent overfitting, as it randomly drops nodes in the CNN during training, preventing the network from becoming too dependent on certain nodes and encouraging it to learn more from the generalised features.

After every convolutional block, Batch Normalisation is applied to balance the convergence speed and stability. The data is then flattened from 3D tensors to a 2D tensor for the joint classifier layer. This layer contains 150 classes because it includes the combination of strings and fret positions: $6(\text{strings}) \times 25(\text{fret}) = 150(\text{classes})$

The model parameters were optimised using the Adam optimiser (adaptive learning rate) with an initial learning rate of 0.001 for efficient convergence. Parameter updates are made from standard backpropagation. Reduce Learning Rate on Plateau (ReduceLROnPlateau) scheduler is used to lower the learning rate if the validation does not improve for 3 consecutive epochs. Lastly, to prevent overfitting and promote generalisation, the training loop monitors the lowest validation loss and utilises model check-pointing to save only the best-performing model's weights to disk. The target is converted to a single integer index and minimised using Cross Entropy Loss. The current version of the model was trained on a subset of 5,000 audio files from the SynthTab dataset over 5 epochs. This constraint was necessary due to computational challenges with running the deep learning models locally.

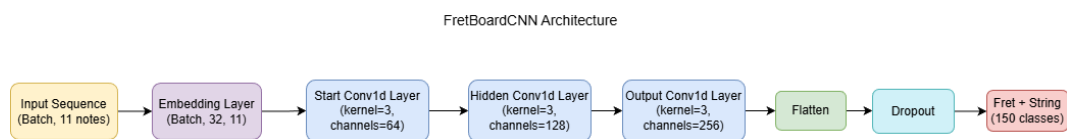


Figure 2: Process of CNN

3.6 Tablature Generation

The requirement for the tab generation currently is to create an ASCII tab based on the information provided by fretboard mapping, as there is no current UI for the AMT. A command line interface (CLI) was developed to facilitate testing and to verify the output accuracy. The current approach does not take into account the possible guitar techniques.

4 Progress and Evaluation

4.1 Pitch Estimation Evaluation

One of the data sets that is being used to test the performance of the audio-to-MIDI pipeline is GuitarSet: a data set designed for guitar transcription, aiming to address the challenge of the polyphonic factor of audio tracks. Consisting of 3 hours of content in total, picking up audio through a hexaphonic pickup stores an output of the separate channels for each of the six guitar strings. The current evaluation uses the mono-microphone setup instead. This mono-microphone setup was chosen over hexaphonic setup to evaluate real-world performance, as most users will upload audio which will not be from a specialised hexaphonic pickups. This will provide a more realistic assessment of the system’s applicability. Metrics tracked of the number of notes found that were correct and the notes that did not match the annotation pitch. Calculating metrics including precision, accuracy and recall using `mir_eval` [24]. The results outline the effect of pre-processing degrading the performance of the pitch detection model, likely because Basic Pitch expects noisy audio rather than a cleaned result. For this reason, the pre-processing will be removed from the pipeline. The post-processing improves the performance of the model and removes additional notes when mapping and will be included in the final version.

Configuration	Precision	Recall	F1-Score	Predicted Notes
Baseline (Basic Pitch)	0.806	0.853	0.821	62659
Pre-processing	0.794	0.687	0.716	46256
Post-processing	0.877	0.812	0.833	52860
Full Pipeline	0.859	0.642	0.711	38396

Table 1: Performance comparison of pipeline configuration. Note: The Ground Truth of GuitarSet contains 62,476 actual notes.

4.2 Fretboard Evaluation

To evaluate the accuracy of the models, the GuitarSet dataset was utilised as a ground truth. One of the challenges when comparing the actual notes (dataset result) and the predicted notes (model results) is the challenge of aligning to the onset time. To address this, a time threshold was used, and the time resembled the closest to the onset time of the actual note, avoiding issues of incorrect matching by implementing greedy matching algorithm. The note is classified as a True Positive (Exact Match) only if the Pitch, Onset Time, String Index, and Fret Index all matched the ground truth.

Metrics of Average String Error and Average Fret Error were calculated to quantify the distance between the predicted and actual position.

To ensure a fair comparison and prevent data leakage, all models were evaluated on the same test split of the GuitarSet dataset. As shown in Table 2, the fretboard CNN significantly outperformed the baseline. These results highlight the data efficiency of the proposed sequential pipeline. While the end-to-end TabCNN struggles to generalise from the limited size of the training set, the model specifically fails to detect note events, predicting only 2898 notes out of the 8715 ground truth notes. In contrast, the modular approach leveraged the capabilities of Basic Pitch, allowing the Fretboard CNN to focus exclusively on the positional mapping.

Model	Accuracy (%)	Avg. String Error	Avg. Fret Error
Simple Heuristic	49.6	0.81	3.88
Fretboard CNN	60.3	0.43	2.01
TabCNN	14.0	0.98	4.87

Table 2: Fretboard Mapping Evaluation. Accuracy denotes the percentage of notes mapped to the exact correct string and fret.

5 Project Management and Reflection

5.1 Project Plan

An agile process was proposed for the management of the project, similar to any other software development. Adopting an agile methodology through weekly supervisory meetings, which ensures alignment with the project’s core aims.

A major issue which the project management Gantt chart faces is deadlines for the submissions being inaccurate, requiring to update the Gantt chart to take into account the deadline being earlier than expected, requiring the re-prioritisation of critical path tasks. Additional tasks have been added to ensure the project can be completed.

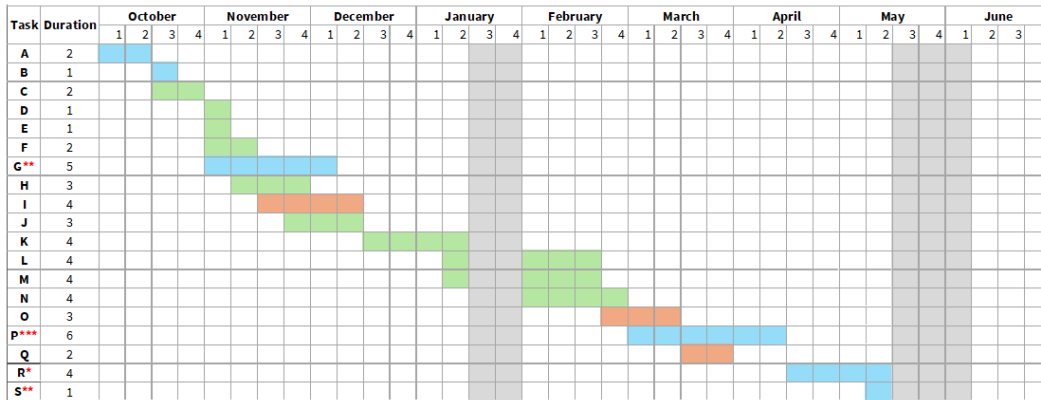


Figure 3: Updated Gantt Chart

ID	Task Description	Start	End	Duration
A	Complete Project Proposal	Oct W1	Oct W2	2 weeks
B	Complete Ethics and Data Management Plan	Oct W3	Oct W3	1 week
C	Foundation & Prototyping (Audio Loading, Basic Pitch)	Oct W3	Oct W4	2 weeks
D	Basic Tab Generation (Single Notes & Simple Melodies)	Oct W4	Oct W4	1 week
E	Multi Track Audio Separation	Nov W1	Nov W1	1 week
F	Improve Tab Generation (Chords, Polyphonic Audio)	Nov W1	Nov W2	2 weeks
G	Write & Submit Interim Report (10% of grade)	Nov W1	Dec W1	5 weeks
H	Implement Sequential Model (CNN Mapping)	Nov W2	Nov W4	3 weeks
I	Model Evaluation	Nov W3	Dec W2	4 weeks
J	Develop End-to-End Model	Nov W4	Dec W2	3 weeks
K	Guitar Technique Classification	Dec W3	Jan W2	4 weeks
L	User Interface Development	Jan W2	Mar W2	4 weeks
M	Automatic Audio Segmentation Implementation	Jan W2	Mar W1	4 weeks
N	Export Functionality for File Format	Feb W1	Mar W1	4 weeks
O	User Testing & Evaluation Study	Feb W4	Mar W3	3 weeks
P	Write Final Dissertation (75% of grade)	Mar W1	Apr W2	6 weeks
Q	Final Code Polish & Demo Video Creation	Mar W3	Mar W4	2 weeks
R	Prepare Presentation & Demo for Assessment	Apr W3	May W2	4 weeks
S	Final Presentation/Demonstration (15% of grade)	May W2	May W2	1 week

Figure 4: Updated Timeline

The project is also being tracked using the DevOps tool, Atlassian Jira. The project management tool has a stronger focus on individual tasks compared to the broad overview of the Gantt chart. Large scale requirements have been set as Epics and problems have been broken down into tasks, improving the modularity of the development and creating a measurable progress tracker

5.2 Meetings Records

During the initial stages of the project, weekly supervisory meetings were conducted, where ideas were exchanged on how to improve the AMT system. Meeting records were developed to ensure supervisor feedback was implemented in the code. This includes scope refinement and aims to ensure feasibility within the time constraints. A key outcome was prioritisation

on transcribing standard tuning. This decision is required due to the limited dataset related to different tunings, allowing the model to be evaluated without introducing additional complexity with the various instrument configurations.

Meeting Record: Progress & Feedback
Agenda: Data Management Plan, Ethics Form, Research Review.
Progress Update: • Started work with <i>Basic Pitch</i>. • Issue: Fretboard fails to map correctly.
Key Supervisor Notes: • Audio Processing: Focus on "peaks and valleys" in raw audio. use smoothing/grouping. Test with guitar scales. • Tools to Check: Google DDSP [25], PureData [26]. • Future: Handle tracks with FX; decide between automated notation vs. user-editable tabs.
References: Serra (DSP Course) [27].

5.3 Contributions and Reflections

A retrospective analysis of the project timeline reveals that the testing phase required more time than initially allocated . Due to the 70/50 credit weighting distribution, and this would allow sufficient time in the second semester to make the project my main priority, as it will be 50 split.

During this semester, extensive research was conducted in-depth regarding the topic of AMT, where I was able to display my findings of the different approaches to automating the process of tab generation.

The main priority during the second term would be to continue improving the accuracy of the tab generation. This would require carrying out the user testing with users validating the performance of the tool and the improvements that would need to be made. One of the key features that will need to be implemented is the ability to recognise guitar techniques. With guitar techniques such as palm mutes and slide-ons, the AMT of this project would be more versatile compared to other available solutions. Another feature that needs to be implemented is the segmentation of the guitar tabs. The purpose of this feature is to split the tabs into parts, like verse and chorus, to make it easier for a user to practice and learn to play the tab. Finally, for the users to be able to access the tool, Future work will focus on the development of user interface, where the users would be able to upload the song and have features such as the option to export to different formats, display the tab generated and give the option for users to update the tab.

5.4 Legal, Social, Ethical and Professional Issues

This project follows strict legal and ethical responsibilities that need to be taken in order to follow the right standards. For instance, it was important to ensure the dataset that was used had not been available in the repository, so as to follow the rules regarding copyright of the dataset that was being used. Another related case is when access to GOAT dataset was requested, due to the dataset containing covers of popular songs. Another case that needs to be aware of is if users upload copyrighted songs to the tool, this would generally not be an issue, as currently the system does not download the song, but it might need to be taken into consideration if reinforcement learning is implemented to improve the model's performance by using the audio the user provided. Similar methods of reinforcement learning have been applied by models MusicRL [28] to align music generation with human preference, while high-dimensional text-to-audio models like MusicLM [29] highlight the growing capabilities and copyright complexities of generative AI. An ethical challenge that the AMT tool faces is the tab generated might not be accurate to the original recording which could lead to users' learning a song incorrectly, to mitigate this risk the system will explicitly include disclaimers regarding potential transcription inaccuracies.

The current version faces a challenge with accuracy, which may lead to some users receiving incorrect tabs. One way to face this problem will be to allow users to amend the tabs that are being generated so they can fix any errors that were made, working well as a practice tool. To ensure users can identify cases where there are errors in the tabs, add the option to view the confidence for users to detect uncertainty in outputs. Another case that must be considered is the case of bias in the model from being trained for a specific genre or style of play (fingerstyle, strumming, and tapping). The model should be trained on a diverse training set to reduce bias. As an automated tool, it is not designed to replace professional transcribers but rather to support individual practice sessions, acknowledging that it cannot provide perfect transcriptions across all musical contexts. For the project one of the concerns with machine learning models include energy consumption and the carbon footprint, to manage this situation priority has been placed on using lightweight models such as Basic pitch and also building models which are computationally cheaper than other option like 1D temporal CNN rather than using transformers like fretting transformer [7]. To make the tool physically accessible for users, the heuristic model was designed using Manhattan distance to minimise users' hand movement, thereby preventing users from injuring their hands when practising.

5.5 Acknowledgements

I am grateful for access to the University of Nottingham's Ada HPC service for the computational resource to train the models. I am also grateful for the access to datasets that were used during this project, including GuitarSet, SynthTab and GOAT.

Bibliography

References

- [1] M. McVicar, A. Pritchard, Y. Sutter, and P. McVicar, “Guitar tab mining, analysis and ranking,” in *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, 2011, pp. 341–346.
- [2] G. Waddell and A. Williamon, “Technology use and attitudes in music learning,” *Frontiers in ICT*, vol. 6, p. 11, 2019.
- [3] E. Benetos, S. Dixon, D. Giannoulis, H. Kirchhoff, and A. Klapuri, “Automatic music transcription: Challenges and future directions,” *Journal of Intelligent Information Systems*, vol. 41, no. 3, pp. 407–434, 2013.
- [4] R. M. Bittner, J. J. Bosch, D. Rubinstein, G. Meseguer-Brocal, and S. Ewert, “A lightweight instrument-agnostic model for polyphonic note transcription and multipitch estimation,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 781–785.
- [5] C. Kehling, J. Abeßer, C. Dittmar, and G. Schuller, “Automatic tablature transcription of electric guitar recordings by estimation of score- and instrument-related parameters,” in *Proceedings of the 17th International Conference on Digital Audio Effects (DAFx-14)*, 2014.
- [6] M. Kaliakatsos-Papakostas, G. Bastas, D. Makris, D. Herremans, V. Katsouros, and P. Maragos, “A machine learning approach for midi to guitar tablature conversion,” in *Proceedings of the 19th Sound and Music Computing Conference (SMC)*, 2022, pp. 192–199.
- [7] A. Hamberger, S. Murgul, J. Schmidt, and M. Heizmann, “Fretting-transformer: Encoder-decoder model for midi to tablature transcription,” in *Proceedings of the 50th International Computer Music Conference (ICMC)*, 2025.
- [8] A. Wiggins and Y. Kim, “Guitar tablature estimation with a convolutional neural network,” in *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, 2019, pp. 284–291.
- [9] M. Good, “Musicxml: An internet-friendly format for sheet music,” in *XML Conference and Exposition*, 2001.
- [10] Y. Zang *et al.*, “Synthtab: Leveraging synthesized data for guitar tablature transcription,” *arXiv preprint arXiv:2404.00000*, 2024.
- [11] J. Loth, P. Sarmiento, S. Sarkar, Z. Guo, M. Barthet, and M. Sandler, “Goat: A large dataset of paired guitar audio recordings and tablatures,” in *Proceedings of the 26th International Society for Music Information Retrieval Conference (ISMIR)*, 2025.
- [12] M. Mauch and S. Dixon, “pyin: A fundamental frequency estimator using probabilistic threshold distributions,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 659–663.

- [13] A. de Cheveigné and H. Kawahara, “YIN, a fundamental frequency estimator for speech and music,” *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002.
- [14] J. W. Kim, J. Salamon, P. Li, and J. P. Bello, “Crepe: A convolutional representation for pitch estimation,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 161–165.
- [15] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [16] J. Gardner, C. Hawthorne, I. Simon, E. Manilow, and J. Engel, “Mt3: Multi-task multitrack music transcription,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [17] X. Riley, D. Edwards, and S. Dixon, “High-resolution guitar transcription via domain adaptation,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024.
- [18] T.-W. Su, Y.-P. Chen, L. Su, and Y.-H. Yang, “TENT: Technique-embedded note tracking for real-world guitar solo recordings,” *Transactions of the International Society for Music Information Retrieval*, vol. 2, no. 1, pp. 15–28, 2019. [Online]. Available: <https://doi.org/10.5334/tismir.23>
- [19] Q. Xi, R. M. Bittner, J. Pauwels, X. Ye, and J. P. Bello, “Guitarset: A dataset for guitar transcription,” in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 453–460.
- [20] J. Paulus, M. Müller, and A. Klapuri, “State of the art in music segmentation and structure analysis,” *EURASIP Journal on Advances in Signal Processing*, vol. 2010, pp. 1–13, 2010.
- [21] O. Nieto and J. P. Bello, “Systematic exploration of computational music structure research,” in *Proceedings of the 17th International Society for Music Information Retrieval Conference (ISMIR)*, 2016, pp. 547–553.
- [22] K. Ullrich, J. Schlüter, and T. Grill, “Boundary detection in music structure analysis using convolutional neural networks,” in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, 2014, pp. 417–422.
- [23] J. B. Smith, J. A. Burgoyne, I. Fujinaga, D. De Roure, and J. S. Downie, “Design and creation of a large-scale database of structural annotations,” in *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, 2011, pp. 555–560.
- [24] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis, “mir_eval: A transparent implementation of common mir metrics,” in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, 2014.
- [25] J. Engel, L. Hantrakul, C. Gu, and A. Roberts, “Ddsp: Differentiable digital signal processing,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [26] M. Puckette, “Pure data: another integrated computer music environment,” in *Proceedings of the Second International Computer Music Conference*. Citeseer, 1996, pp. 37–41.
- [27] X. Serra and J. O. Smith III, “Audio signal processing for music applications,” Coursera, 2019, accessed: 2025-12-06. [Online]. Available: <https://www.coursera.org/learn/audio-signal-processing>

- [28] G. Manco, E. Benetos, and S. Dixon, “Musicrl: Aligning music generation to human preferences,” *arXiv preprint arXiv:2402.04229*, 2024.
- [29] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzett, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi *et al.*, “Musiclm: Generating music from text,” *arXiv preprint arXiv:2301.11325*, 2023.